

**16/32-bit Linear
Executable Module
Format 0:32
and
Windows Virtual Device
Driver (VxD) Model**

Revision 1.0

Contents

Introduction	3
Linear Executable File Format	4
Program Header	6
Target CPU Type	8
Target OS Type	8
Module Flags	8
Module Pages	10
Object Table	11
Resource Table	12
Entry Points Table	14
Entry Bundle for a Target object	15
Unused Entry	16
16-bit Entry	17
16-bit Call Gate Entry	17
32-bit Entry	18
Forwarder Entry	18
Resident Names Table	19
Non-Resident Names Table	20

Introduction

This document is intended to describe the interface that is used by language translators and generators as their intermediate output to the linker for the Microsoft OS/2 2.0 operating system. The linker will generate the executable module that is used by the loader to invoke the *.EXE and *.DLL programs at execution time.

This material was compiled from Microsoft OS/2 2.0 SDK sources, leaked Windows Debugger (short. windbg), Open Watcom documents and was written by Alexey Tolstopyatov the Tomsk State University student

Linear Executable File Format

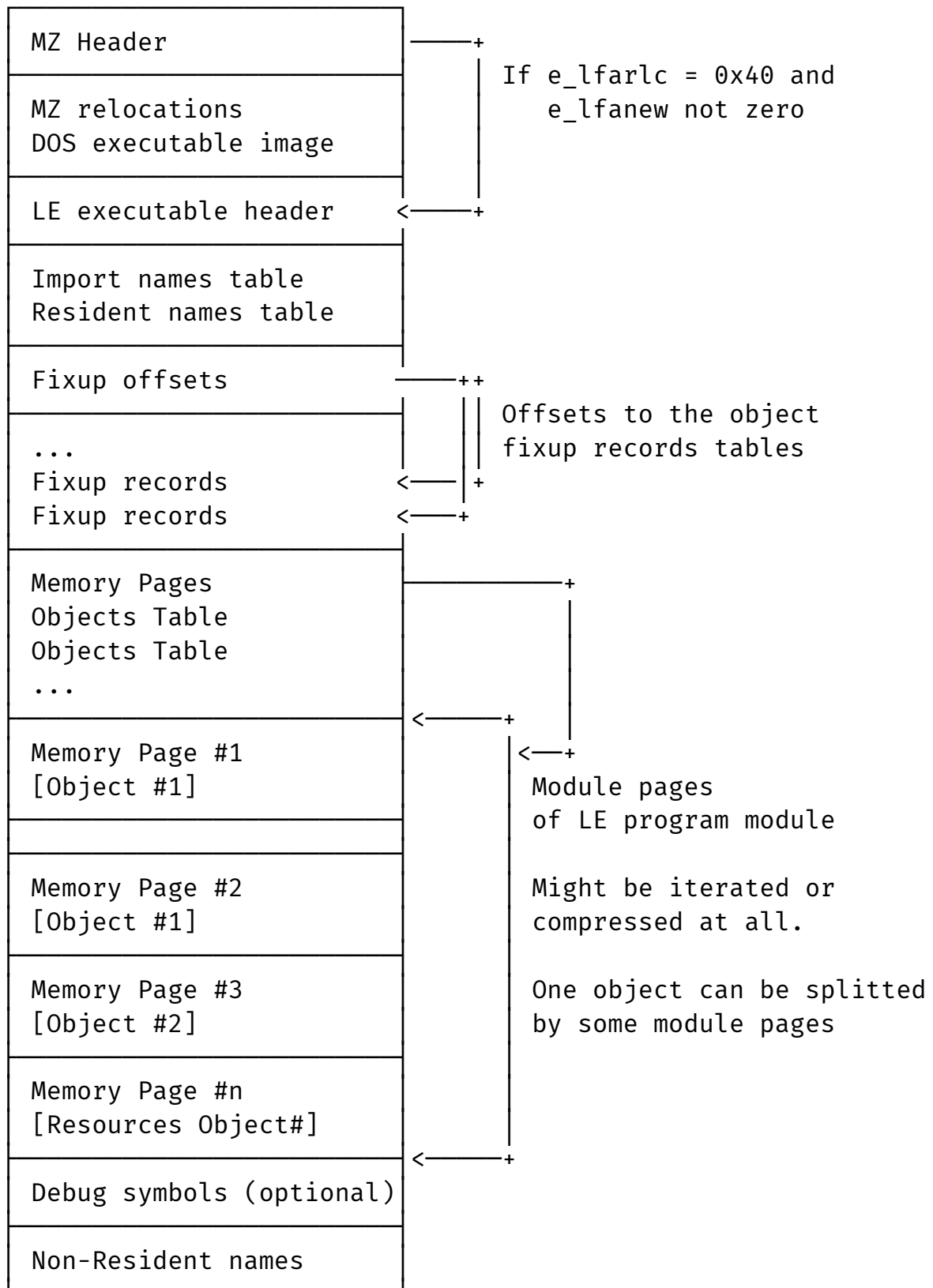
Linear Executable (short. LE) is file format for executable code designed for 16/32-bit protected mode operating systems. Originally used by the Microsoft OS/2 2.0 operating system, served as the file format for Virtual Device Drivers (VxD) in early versions of Windows, and adopted by various DOS extenders. Microsoft Windows 3.x and 9x used special archive format for VxD drivers – W3 and W4 executable containers.

Do not use IBM's experience to determine the LE format correctly. The standard IBM OS/2 executable module format is a logical development of this model, which is not backward compatible!

Same with the New Executable format, first bytes of file belong to the Mark Zbikowski executable header. After those header bytes the DOS compatible program image starts. The field by the 0x3C offset still hold an offset from the beginning of file to the next protected-mode executable image.

DOS 2.0 compatible part will be skipped in this manual. First 2 bytes after `e_lfanew` long offset will be ASCII signature [0x45, 0x4C]. It has not been experimentally confirmed that the byte order affects the signature, however, I allow different byte order. That's why, is better to check LE and EL ASCII signatures.

At the next page presented file layout (see Listing 2)



Listing 2: Linear executable file layout

Despite the fact that LE format very similar with child – LX format, don't apply LX module tables to LE module format.

Since the IBM modification of linear executable module format, DOS 2.0 compatible program might be removed from file. So and *.EXE/DLL/SYS files will contain only LX program part.

Program Header

At this table presented full map of linear executable header

Offset	Size	Name	Description
0x00	2	e32_magic	Magic ASCII (0x454C)
0x02	1	e32_border	Byte ordering
0x03	1	e32_worder	Word ordering
0x04	4	e32_level	Format level (0 for initial version)
0x08	2	e32_cpu	CPU type
0x0A	2	e32_os	Operating system type
0x0C	4	e32_ver	Module version number
0x10	4	e32_mflags	Module flags
0x14	4	e32_mpages	Number of pages in module
0x18	4	e32_startobj	Object number for instruction pointer
0x1C	4	e32_eip	Extended instruction pointer
0x20	4	e32_stackobj	Object number for stack pointer
0x24	4	e32_esp	Extended stack pointer (offset within object)
0x28	4	e32_pagesize	Page size in bytes (default 4096)
0x2C	4	e32_lastpagesize	Size of last page in bytes
0x30	4	e32_fixupsize	Fixup section size in bytes
0x34	4	e32_fixupsum	Fixup section checksum
0x38	4	e32_ldrsize	Loader section size in bytes
0x3C	4	e32_ldrsum	Loader section checksum
0x40	4	e32_objtab	Offset of object table from start of LE header
0x44	4	e32_objcnt	Number of objects in module
0x48	4	e32_objmap	Offset of object page map

Offset	Size	Name	Description
0x4C	4	e32_itermap	Offset of object iterated data map
0x50	4	e32_rsrctab	Offset of resource table
0x54	4	e32_rsrcnt	Number of resource entries
0x58	4	e32_restab	Offset of resident name table
0x5C	4	e32_enttab	Offset of entry table
0x60	4	e32_dirtab	Offset of module directive table
0x64	4	e32_dircnt	Number of module directives
0x68	4	e32_fpagetab	Offset of fixup page table
0x6C	4	e32_frextab	Offset of fixup record table
0x70	4	e32_impmod	Offset of import module name table
0x74	4	e32_impmodcnt	Number of entries in import module name table
0x78	4	e32_impproc	Offset of import procedure name table
0x7C	4	e32_pagesum	Offset of per-page checksum table
0x80	4	e32_datapage	Offset of enumerated data pages
0x84	4	e32_preload	Number of preload pages
0x88	4	e32_nrestab	Offset of non-resident names table
0x8C	4	e32_cbnrestab	Size of non-resident name table in bytes
0x90	4	e32_nressum	Non-resident name table checksum
0x94	4	e32_autodata	Object number for automatic data object
0x98	4	e32_debuginfo	Offset of debugging information
0x9C	4	e32_debuglen	Length of debugging information in bytes
0xA0	4	e32_instpreload	Number of instance pages in preload section
0xA4	4	e32_instdemand	Number of instance pages in demand section
0xA8	4	e32_heapsize	Size of heap for 16-bit applications
0xAC	24	e32_res3	Reserved bytes to pad structure to 196 bytes

Total size of LE header – 196 bytes (or 0xC4). All offsets are from the start of it.

Differences between LE and LX headers starts here already. Field by the 0x2C offset from the beginning of header doesn't store module pages alignment (short. e32_pageshift)

This difference is one of the key ones in the comparison. Further more. In the following sections, the layout of memory pages will be discussed, and this field will help.

Target CPU Type

Value	CPU Type
0x01	Intel 80286 or upwardly compatible
0x02	Intel 80386 or upwardly compatible
0x03	Intel 80486 or upwardly compatible
0x04	Intel Pentium or upwardly compatible

Table 1: CPU types. Values are stored in the e32_cpu field.

Target OS Type

Value	Description
0x00	Any “new format” OS
0x01	OS/2 2.0+
0x02	Windows
0x03	DOS 4.0
0x04	Windows/386

Table 2: OS types. Values are stored in the e32_os field.

Module Flags

This information bases on exe386.h header from MS OS/2 SDK. The module flags field e32_mflags divided on high and low WORD bitmasks. Empty cells at the high and low WORD columns mean zero value.

Bit(s)	Mask	Description
0	0x0001	Reserved
1	0x0002	Reserved
2	0x0004	Per-Process Library Initialization
3	0x0008	Reserved
4	0x0010	No Internal Fixups for Module
5	0x0020	No External Fixups for Module
6-7	0x00C0	Reserved
8	0x0100	Incompatible with PM Windowing
9	0x0200	Compatible with PM Windowing
10-11	0x0C00	Uses PM Windowing API
12	0x1000	Reserved
13	0x2000	Module not Loadable
14	0x4000	Reserved
15	0x8000	Library Module

Table 3: Low word of module flags.

Bit(s)	Mask	Description
0-14		Reserved
15	0x8000	.DLL module
16	0x10000	Protected memory library module
17	0x20000	Device driver
?	0x38000	Virtual DLD Driver
18	0x40000	Reserved
19-31		Reserved

Table 4: High word of e32_mflags

Then next listing demonstrates a way to define module flags from the given parsed header, (e32_mflags filled already).

```

uint32_t mflags = 0x00000212;
uint32_t pmcompat = mflags & 0x00000200;
uint32_t isdll = mflags & 0x00000800;

if (pmcompat) printf("Compatible with OS/2 PM");
if (isdll) printf("Library module");

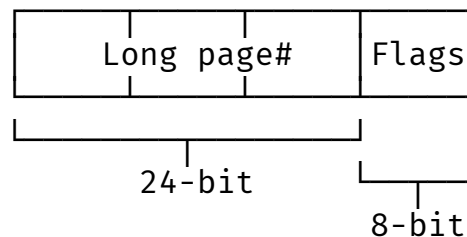
```

Listing 3: Resolving module flags bitmask routine

Module Pages

Module pages concept defines the special “physical sections” of program text and data. At the moment of format revision 0:32 by IBM and Microsoft, module pages records are represent an array of 32-bit records.

On the next listing presented a single module page record.



Listing 4: Module page record layout

Numbering of module pages starts from one. Count of module pages contains in the executable header in the e32_mpages field. For each module page except last page the e32_pagesize is valid. Last module page have size which equal e32_lastpagesize value.

After the linkage the Module pages always aligned by the paragraph (0x10). To compute file offset to the page start, make sure that you know

- Long Page index;
- Page is valid (Flags set to zero);
- Page size (e32_pagesize);
- Offset to enumerated datapages (e32_datapage);

```
page_offset := e32_datapage + (page# - 1) * e32_pagesize;
```

Per-page attributes or Flags field presented in the previous listing.

Byte	Description
0x00	Valid Physical Page
0x01	Iterated page
0x02	Invalid page
0x03	Zeroed page
0x04	Pages range

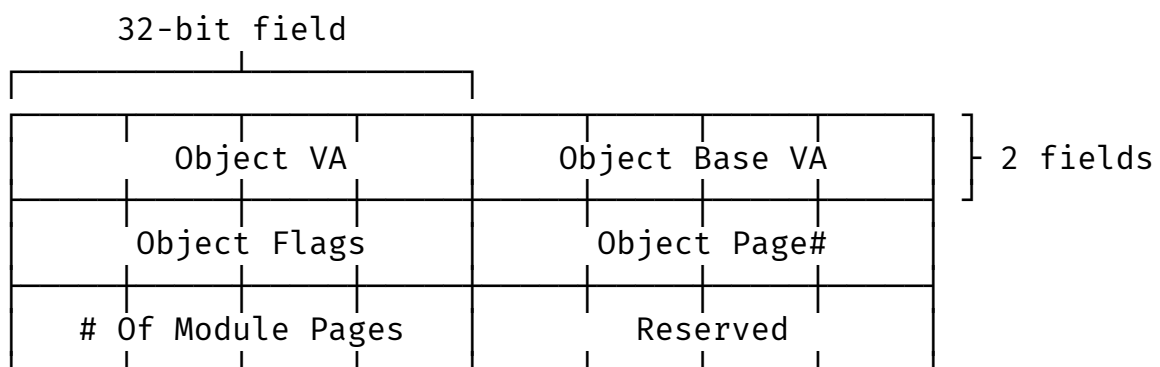
Table 5: Per-page attributes

Object Table

From the point of view of file, objects in the linear executable represents a reserved range of module pages where some text or data could be placed.

To find out which module pages reserved for which object, the file offset of the object table `e32_objtab` and `e32_objcnt` are read from the program header and entire table is reviewed.

Objects table is just an array of object records. Object record presented next



Listing 6: Object record layout

On this listing the DWORD fields ordering starts from upper left and reads till the least right field at the bottom.

Per-Object flags is a bitmask field which may contain bits which defined in the next table

Word	Description
0x0001	Readable Object
0x0002	Writable Object
0x0004	Executable Object
0x0008	Resource Object
0x0010	Object is discardable
0x0020	Object is shared
0x0040	Object has preload pages
0x0080	Object uses invalid pages

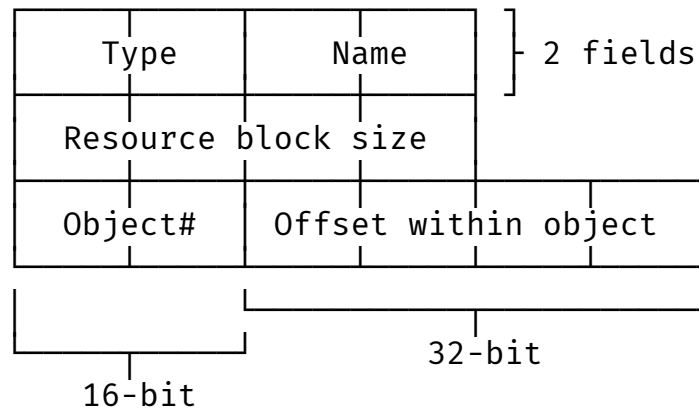
Table 6: Object flags (low byte)

Word	Description
0x0100	Object is permanent and swappable
0x0200	Object is permanent and resident
0x0300	Object is resident and contiguous
0x0400	Object is permanent and long locable
0x0600	Object is nonpermanent – should be
0x0700	Object type-mask
0x1000	16:16 alias required (80x86 specific)
0x2000	Big bit setting (80x86 specific)
0x4000	Object is conforming for code
0x8000	Object I/O Privileges

Table 7: Object flags (High byte)

Resource Table

The resource table is an array of resource table entries. Each resource table entry contains a type and name. These entries are used to locate resource objects contained in the Object table.



Listing 7: Resource Table record

The number of entries in the resource records is defined by the Resource Table Count located in the linear executable header. More than one resource may be contained within a single object.

On the next table presented available resource types for Microsoft OS/2 2.0

Byte	Description
0x01	Mouse pointer shape
0x02	Bitmap
0x03	Menu
0x04	Dialog
0x05	Strings Table
0x06	Font Directory
0x07	Font
0x08	Accelerator Table
0x09	Binary Data
0x0A	Message Tables (embedded *.msg file)
0x0B	Dialog include file name
0x0C	Key to vkey tables
0x0D	Key to UGL tables
0x0E	Glyph to character tables
0x0F	Screen display information
0x10	Function key area (short form)
0x11	Function key area (long form)
0x12	Help table
0x13	Help subtable
0x14	DBCS uniq/font driver directory
0x15	DBCS uniq/font driver

Table 8: Object flags (low byte)

Entry Points Table

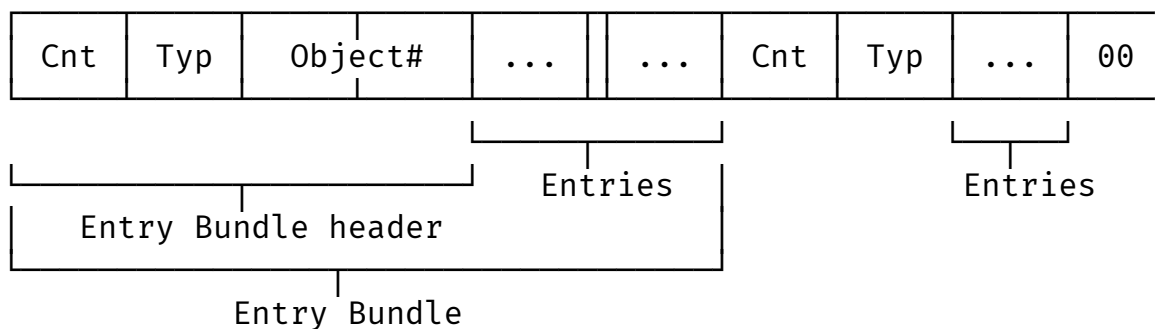
Entry points table (short. Entry Table) is very complex structure in the flat executable file format. Same with the segmented executables (short. NE) from OS/2 1.x and Windows 1.x, the entry table contains entry bundles.

Despite the fact that many different sources indicate the existence of only two types of entry bundles, the `exe_vxd.h` of leaked Windows debugger and `exe386.h` from Microsoft OS/2 2.0 SDK determines next types of entry point:

- Unused Entry;
- 16-bit Entry;
- 16-bit Callgate Entry;
- 32-bit Entry;
- Forwarder Entry;

```
type := typ & 0x7F;
has_add_typinfo := (typ & 0x80) != 0;
```

The bundle of entry points (short. Entry Bundle) contains data like an `object#` and type for all entry points which store in the current bundle. Next listing is a try to visualize a flat memory model of imaginary entry table.

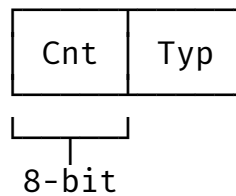


Listing 9: Entry Table layout

Entry Table starts with the BYTE which defines count of entry points in the given bundle. If entries count equals zero, this is a mark of the end of an Entry Table.

Entry Bundle for a Target object

Entry bundle header differs for other types of entries. If entry points bundle contains Forwarder entries – the target `object#` for entries in bundle ignores (doesn't exist at all). Linker and operating system sets `Object#` to zero.

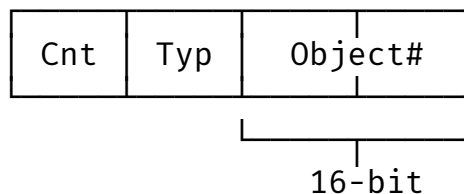


Listing 10: Forwarder or Unused Entry Bundle header

If object# exists and reads by the operating system loader, there're exist two ways.

- object# = 0 then offset of each entry point in bundle are absolute file offset;
- object# not zero then each entry point (depending on the target object flags) have a virtual address!

Use Object Table and Module Pages Table to resolve the position of the entry point.



Listing 11: Entry Bundle header

All what is going next after header will be present an array of entry points with the type declared in the entry bundle header. Length of this array defines by the count field in the header (short. Cnt).

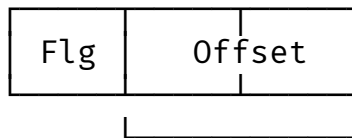
Entry Table represents a module exports which will be resolved during the another application run-time. Procedure ordinals enumeration starts from 1. Entry Point at the 0-ordinal doesn't exist, but ordinal 0 exists and reserved by module.

Unused Entry

Unused Entry can't have a body instead of other types of enties. This is a way how to place export procedures @1 and @100, for example. Linker skips the count of unused entries and starts new bundle.

16-bit Entry

After Entry Bundle, defined in previous region, follows an array of 16-bit entry points. For 16-bit program code entry point defines like on the next listing.



Look at the target object flags too.
Usually (if object doesn't have invalid pages)
this is an offset from the object start.

Listing 13: 16-bit Entry

Flags BYTE field has a special bitmask.

Byte	Description
0x01	Exported Entry
0x02	Uses Shared .DATA
0xF8	Parameter work mask

Table 9: Object flags (low byte)

16-bit Call Gate Entry

The Intel 286 Call Gate Entry Point type is needed by the loader only if ring-2 segments are to be supported (Object flag bitmask includes IO PRIVILEGIES flag). The I286 Call Gate entries contain 2 extra bytes which are used by the loader to store an LDT callgate selector value



Set to zero in the file.
Stores LDT selector during a run-time.

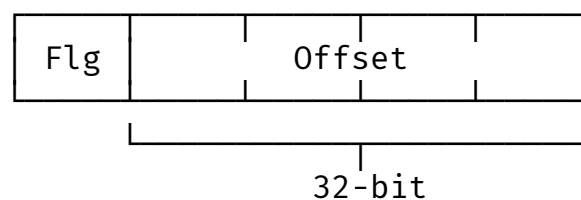
Listing 14: 16-bit Call Gate Entry

The DOSCALL1.DLL from the MS OS/2 stores I286 Call Gate entries. Examples of them will be the famous DosCopy procedure or DosRead/DosWrite procedures.

Flags of the Call Gate entry are the same with the 16-bit entry flags. So, see the Table 9.

32-bit Entry

Same with the 16-bit entries – 32-bit entries relate to objects with USE_32 flag in their bitmasks.



Listing 15: 32-bit Call Gate Entry

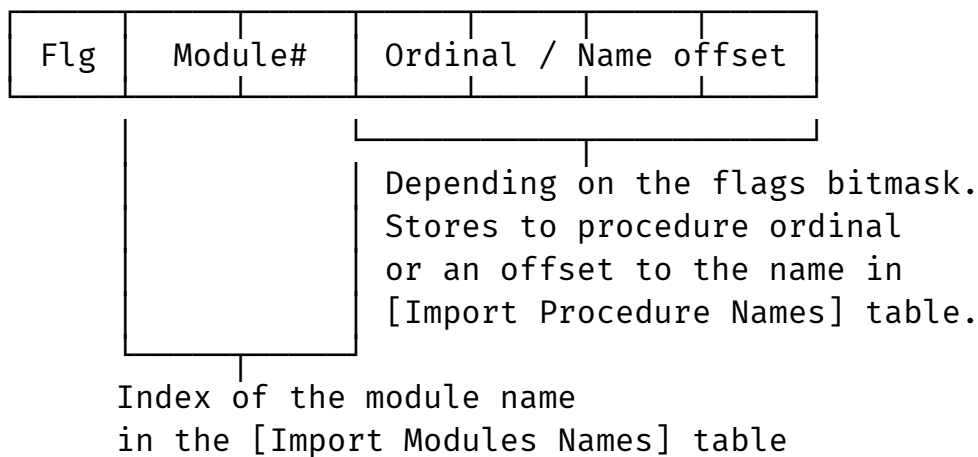
The Flg field contains same flags presented in the Table 9

Forwarder Entry

A Forwarder entry (short. forwarder) is an entry point whose value is an imported reference.

When a load time fixup occurs whose target is a forwarder, the loader obtains the address imported by the forwarder and uses that imported address to resolve the fixup.

A forwarder may refer to an entry point in another module which is itself a forwarder, so there can be a chain of forwarders. The loader will traverse the chain until it finds a non forwarded entry point which terminates the chain, and use this to resolve the original fixup. Circular chains are detected by the loader and result in a load time error.



Listing 17: 16-bit Call Gate Entry

For the forwarder entries the flags definitions are different.

Byte	Description
0x01	Import by ordinal
0xF7	Reserved!

Table 10: Object flags (low byte)

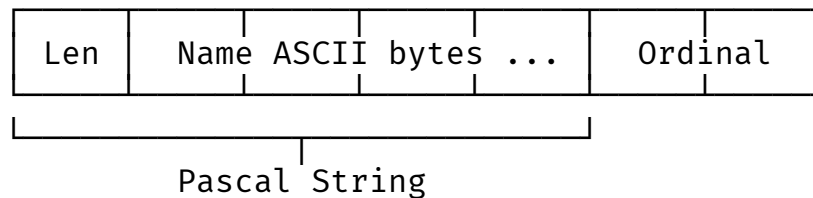
A maximum of 1024 forwarders is allowed in a chain; more than this results in a run-time error.

Forwarders are useful for merging and recombining API calls into different sets of libraries, while maintaining compatibility with applications. *For example, if one wanted to combine MONCALLS, MOUCALLS, and VIOCALLS into a single libraries, one could provide entry points for the three libraries that are forwarders pointing to the common implementation.*

Resident Names Table

Resident names table contains export procedures which must be kept in the system memory during the module run-time. It is intended to contain the exported entry point names that are frequently dynamically linked to by name.

Resident name record consists of procedure ordinal word and pascal string of the procedure name.



Listing 18: Procedure Name record

The end of the table defines by the zero length of the next Pascal String.

Non-Resident Names Table

Non-resident names are not kept in memory and are read from the file when a dynamic link reference is made. Exported entry point names that are infrequently dynamically linked to by name or are commonly referenced by ordinal number should be placed in the non-resident name table.

Non-Resident names Table pointer in the Linear Executable header declares the offset from the start of file! This is a once structure in the LE layout which location defines not from start of program header.

Same with the Resident Names table – it just an array of records (see Listing 18) with undefined count of entries.

The end of the table defines by the zero length of the next Pascal String.